Multiplicacao de Matrizes em Blocos com MPI e OpenBLAS

Resumo

Foram compiladas e testadas as tres versoes do projeto no Lovelace. O teste de corretude com `N=128`, bloco 32 e dois processos terminou com `validation=ok` para `mpi_mm` e `mpi_blas_mm`. A campanha forte produziu 105 medicoes validas: 35 configuracoes, cada uma com tres repeticoes. Tambem foram executadas nove medicoes de escalabilidade fraca.

Os resultados sao reais e estao em `results/raw/` e `results/processed/`.

Ambiente experimental

• Host: `lovelace.ada.cenapad.unicamp.br`.

• CPU reportada: AMD EPYC 7282 16-Core Processor.

• CPUs visiveis na sessao: 1.

• Memoria reportada: 62 GiB.

• Kernel: `4.18.0-553.150.1.el8_10.x86_64`.

• Compilador: GCC 9.4.0, modulo `gcc/9.4.0`.

• MPI: OpenMPI 4.1.1, modulo `openmpi/4.1.1-gcc-9.4.0`.

• CMake: 4.3.1, modulo `cmake/4.3.1`.

• OpenBLAS: modulo `openblas/0.3.21-gcc-9.4.0`; o executavel foi confirmado

com `ldd` usando `libopenblas.so.0`.

• Python: 3.6.8.

• Matplotlib: 3.3.4.

• Launcher usado: `/opt/pub/openmpi/4.1.1/gcc/9.4.0/bin/mpiexec`.

• `OPENBLAS_NUM_THREADS=1` foi definido nos testes MPI+OpenBLAS.

As execucoes foram feitas diretamente no host acessivel por SSH. Nao foi obtida uma alocacao PBS multinode nesta rodada; portanto, os resultados nao devem ser interpretados como comparacao entre nos.

Correcoes realizadas

O CMake original nao propagava o diretorio `include/` para os executaveis MPI. As propriedades de include das bibliotecas `mpi_matmul_core` e `mpi_blas_core` foram alteradas para `PUBLIC`.

Tambem foi corrigido um erro de execucao em `src/mpi_matmul.cpp`: o buffer `gathered` so era alocado quando `--validate` estava ativo, embora `MPI_Gatherv` fosse sempre chamado. Isso causava `SIGSEGV` em qualquer benchmark MPI sem validacao.

Corretude

Comandos executados:

./build/serial_mm --n 128 --block 32 --validate
mpiexec -np 2 ./build/mpi_mm --n 128 --block 32 --validate
OPENBLAS_NUM_THREADS=1 mpiexec -np 2 ./build/mpi_blas_mm --n 128 --block 32 --validate

Resultados:

Versao	Max abs	Max relativo	Validacao (s)	Resultado
serial	0	0	0.0000257	ok
MPI	0	0	0.0003119	ok
MPI + OpenBLAS	7.62939453e-06	0.00470212	0.0003114	ok

O tempo de validacao foi medido depois da computacao e nao foi incluido em `compute_seconds` ou `total_seconds`.

Campanha de desempenho

A campanha forte usou um host, `N=2000, 4000, 6000`, blocos `64, 128, 256` quando aplicavel, processos MPI `1, 2, 4` e tres repeticoes. Para `N=4000` e `N=6000`, foi usado bloco `128` para limitar o custo da campanha. Os benchmarks foram executados sem `--validate`, pois a validacao serial de uma matriz grande nao deve ser parte da campanha de desempenho.

Arquivos gerados:

- `results/processed/measurements.csv`: 105 linhas de medicao.
- `results/processed/statistics.csv`: 35 grupos estatisticos.
- `results/processed/speedup_efficiency.csv`: speedup relativo a serial.
- `results/plots/compute_time_n2000.png`.
- `results/plots/compute_time_n4000.png`.
- `results/plots/compute_time_n6000.png`.

Exemplos de medias de `compute_seconds` para bloco `128`:

N	Serial	MPI+BLAS p=1	MPI+BLAS p=2	MPI+BLAS p=4
2000	0.882474	0.235511	0.117759	0.059574
4000	6.810632	1.893012	0.945933	0.472410
6000	23.145013	7.264287	3.621751	1.609950

Esses valores sao medias das tres repeticoes e devem ser analisados junto com `statistics.csv`, que contem desvio-padrao, minimo e maximo.

Escalabilidade forte

Foi mantido `N` fixo e aumentado o numero de processos entre 1 e 4. A referencia de speedup entre versoes foi a media serial da mesma matriz e do mesmo bloco. O speedup paralelo foi calculado como `T(1)/T(p)` e a eficiencia como `speedup/p`. Os valores completos estao em `results/processed/speedup_efficiency.csv`.

Como os processos foram executados em um host com apenas uma CPU visivel, estes resultados medem principalmente oversubscription e nao escalabilidade de nucleos fisicos. Nao foi feita afirmacao de escalabilidade multinode.

Escalabilidade fraca

Foi usada a regra aproximada `N proporcional a p^(1/3)`, mantendo bloco `128` e a versao MPI+OpenBLAS:

- `p=1, N=2000`;
- `p=2, N=2520`;
- `p=4, N=3175`.

Cada configuracao teve tres repeticoes. Os logs estao no diretorio `results/raw/weak-*` e os dados processados em `results/processed/weak_measurements.csv`.

Reprodutibilidade

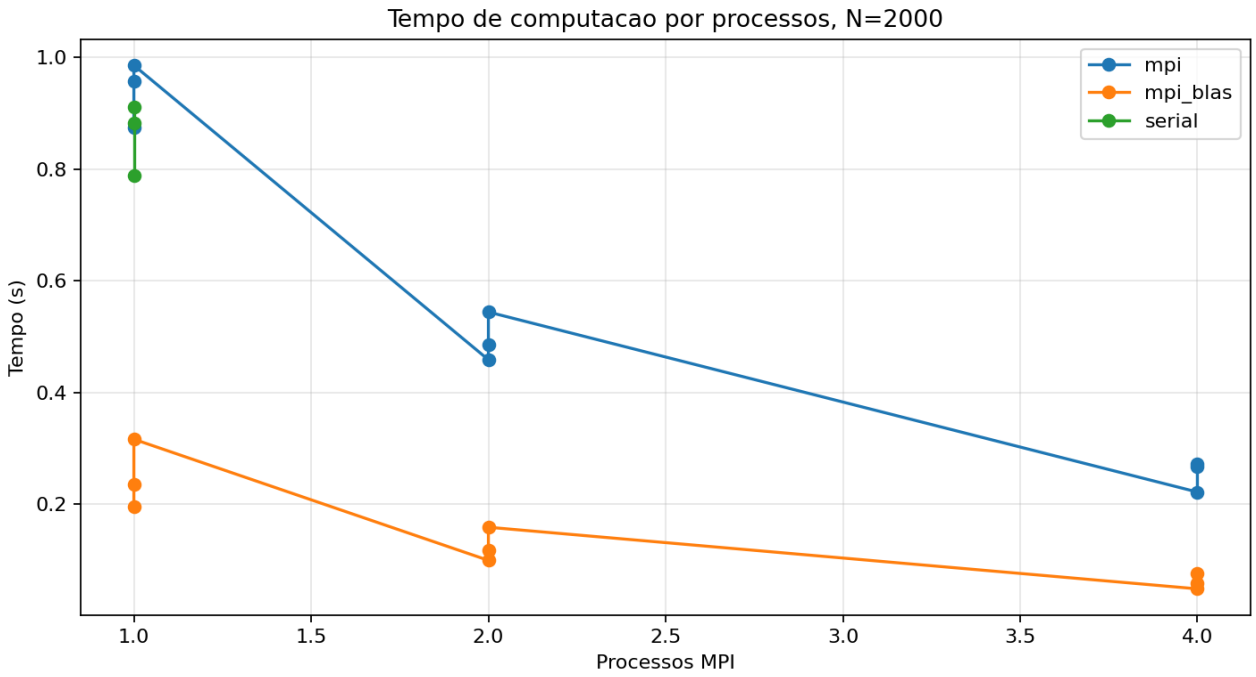
Evidencias preservadas:

- `results/environment/inventory/`;
- `results/environment/bash-syntax.txt`;
- `results/environment/build-final.txt`;
- `results/environment/build-after-mpi-fix.txt`;
- `results/environment/mpi_blas_ldd.txt`;
- `results/raw/correctness/`;
- `results/raw/final-*`;
- `results/raw/weak-*`;
- `results/processed/`;
- `results/plots/`.

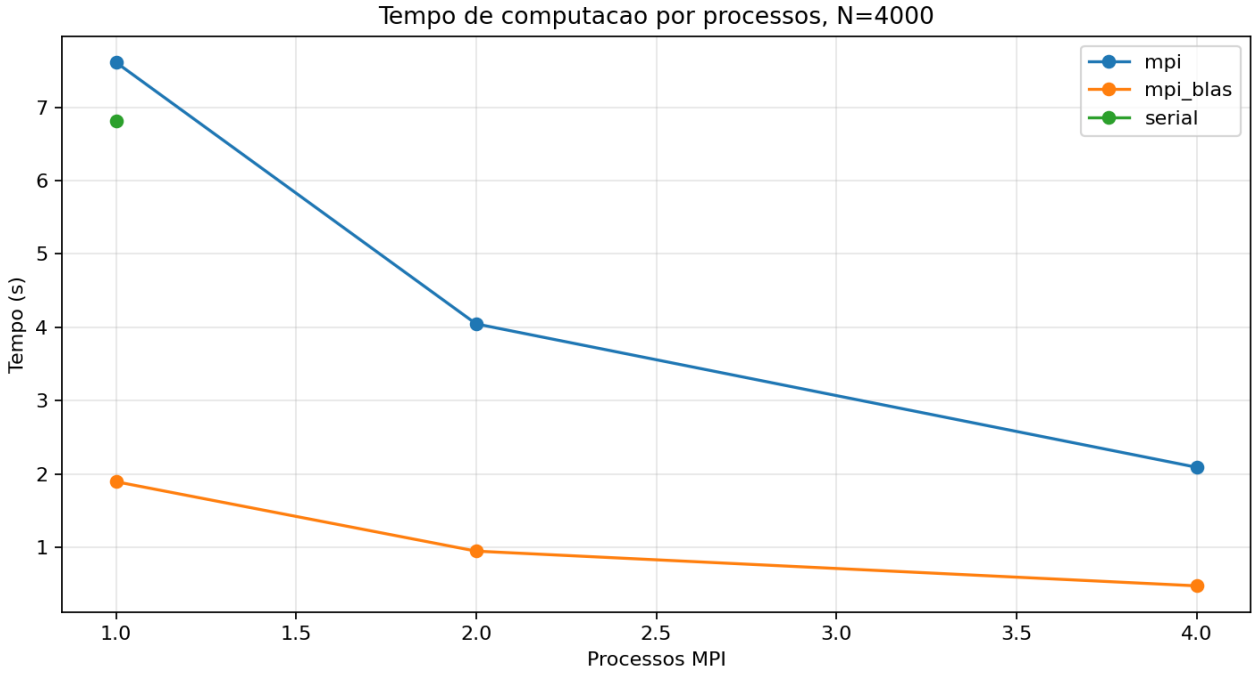
Todos os scripts Bash e PBS passaram por `bash -n`. A campanha multinode e a submissao PBS nao foram concluídas nesta rodada porque a sessao usada expôs uma CPU e nao houve alocacao PBS registrada para este trabalho. Tambem nao foi gerado `report/report.pdf`; o conteudo factual esta neste Markdown.

Graficos de desempenho

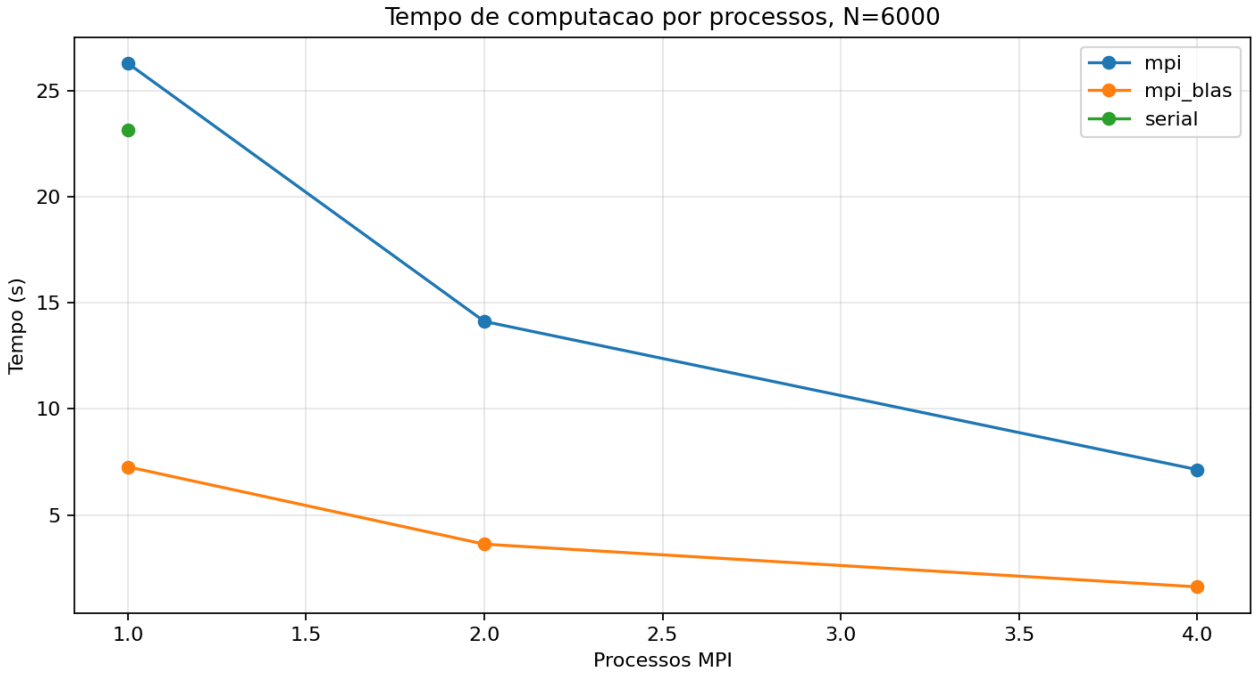
compute_time_n2000.png



compute_time_n4000.png



compute_time_n6000.png


Tempo de computacao por processos, N=6000
mpi
mpi_blas
serial
Tempo (s)
25
20
15
10
5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Processos MPI